Bayesian Statistics, MCMC, & Physics-Informed Neural Networks

Meet 3 Foundations: Parameter Uncertainty & Constrained Machine Learning Models

---

Astronomy Club

July 17, 2026

Meet Overview

This meet bridges the gap between statistical parameter estimation and machine learning by teaching Bayesian inference and Physics-Informed Neural Networks (PINNs) from scratch:

Part 1: Foundations of Bayesian Inference

- Bayes' Theorem, priors, likelihoods, posteriors, and the normalizing evidence challenge.

Part 2: Markov Chain Monte Carlo (MCMC) & Sampling

- Stationary chains, Metropolis-Hastings sampling, and trace diagnostics.

Part 3: Physics-Informed Machine Learning (PIML)

- Soft vs. hard constraints, automatic differentiation, and architectural bounds.

Part 4: Case Study: Supernova Rate Modeling (Task 13)

- Fitting rates using MLE, Metropolis MCMC, and PyTorch PINN parameter recovery.

Part 5: Case Study: Convolved DTD Solvers (Task 15)

- Reconstructing delay time distributions using convolved PyTorch solvers.

Part 1: Foundations of Bayesian Inference

Frequentist vs. Bayesian Probability

To understand parameter estimation, we must first contrast how it defines "probability" compared to classical frequentist statistics:

Frequentist Probability (Objective Limit): Probability is the limiting relative frequency of an event in an infinite sequence of identical, repeatable trials.

- Limitation: Cannot assign a probability to one-off, non-repeatable events (e.g., "What is the probability that this specific galaxy hosts a Type Ia supernova?").

Bayesian Probability (Degree of Belief): Probability is a measure of our state of knowledge or degree of belief given incomplete information.

- Advantage: Allows us to update our beliefs as new data arrives and to quantify uncertainties on constant physical parameters (like supernova rate coefficients A and B).

Bayes' Theorem

Bayes' Theorem is the fundamental equation of Bayesian inference. We derive it directly from the definition of conditional probability:

- The joint probability of two events A and B occurring is:

$$P(A \cap B) = P(A \mid B) \cdot P(B)$$

- By symmetry, we can also express this joint probability as:

$$P(A \cap B) = P(B \mid A) \cdot P(A)$$

- Equating the two right-hand sides:

$$P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$$

- Dividing both sides by $P(B)$ yields Bayes' Theorem:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

The Components of Bayesian Parameter Inference

In scientific parameter estimation, we replace event A with our model parameters θ (e.g., rate coefficients A, B), and event B with our observations D (host galaxy catalog):

$$P(\theta \mid D) = \frac{P(D \mid \theta)P(\theta)}{P(D)}$$

Posterior $P(\theta \mid D)$: What we know about the parameters θ **after** seeing the data D. This is our target probability distribution.

Likelihood $P(D \mid \theta)$: The probability of observing the data D given a specific parameter set θ.

Prior $P(\theta)$: What we knew about the parameters θ **before** seeing the data D (based on previous experiments or physical boundaries).

Evidence $P(D)$: The normalizing marginal likelihood (a constant).

The Normalization Challenge

To ensure the posterior is a valid probability distribution (integrates to 1.0), we divide by the evidence $P(D)$. By the law of total probability, the evidence is the integral of the numerator over the entire parameter space:

$$P(D) = \int_{\theta} P(D \mid \theta) P(\theta) d\theta$$

The Curse of Dimensionality

If we have a single parameter θ, this integral is easy to solve. If we have $d = 10$ parameters, and we divide the parameter range into 100 grid points, we must evaluate:

$$100^{10} = 10^{20} \text{ likelihood calculations}$$

which would take supercomputers years to calculate. For high-dimensional physical models, solving this integral analytically or via simple numerical integration is impossible.

Part 2: Markov Chain Monte Carlo (MCMC) & Sampling

What is Monte Carlo Integration?

If we cannot solve the normalization integral analytically, how do we evaluate the posterior? We use **Monte Carlo Integration** (sampling):

- Instead of evaluating an integral analytically, we draw random independent samples $\{\theta_1, \theta_2, \ldots, \theta_M\}$ from the probability distribution.
- **Example: Estimating π by throwing random darts:**
  1. Draw a circle of radius r inside a square of side $2r$.
  2. Generate random coordinates (x, y) inside the square.
  3. Count the fraction of coordinates that fall inside the circle:

$$\frac{N_{\text{inside}}}{N_{\text{total}}} \approx \frac{\text{Area}_{\text{circle}}}{\text{Area}_{\text{square}}} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4} \implies \pi \approx 4 \cdot \frac{N_{\text{inside}}}{N_{\text{total}}}$$

- By drawing samples from the posterior, we can compute averages, standard deviations, and confidence intervals directly from the sample histograms without ever calculating the normalizer $P(D)$.

What is a Markov Chain?

How do we draw samples from a complex, non-standard posterior distribution $P(\theta \mid D)$? We construct a **Markov Chain**.

The Markov Property (Memorylessness)
A Markov Chain is a sequence of random variables $\{\theta_0, \theta_1, \theta_2, \ldots, \theta_t\}$ where the probability of the next state θ_{t+1} depends **only** on the current state θ_t, not on the history of previous states:

$$P(\theta_{t+1} \mid \theta_0, \theta_1, \ldots, \theta_t) = P(\theta_{t+1} \mid \theta_t)$$

- We start at a random point in parameter space.
- At each step t, we propose a transition to a new state θ_{prop} based on a transition probability that depends only on θ_t.
- If we design the transition rules correctly, the distribution of samples will converge to a unique **stationary distribution** that matches our target posterior $P(\theta \mid D)$.

Bypassing the Normalizer: The Posterior Ratio

The core magic of Markov Chain Monte Carlo (MCMC) is that we do not need to calculate the evidence $P(D)$.

- When deciding whether to move from current state θ_t to proposed state θ_{prop}, we compare their posterior probabilities as a ratio:

$$\frac{P(\theta_{\text{prop}} \mid D)}{P(\theta_t \mid D)} = \frac{\left(\frac{P(D|\theta_{\text{prop}})P(\theta_{\text{prop}})}{P(D)}\right)}{\left(\frac{P(D|\theta_t)P(\theta_t)}{P(D)}\right)} = \frac{P(D \mid \theta_{\text{prop}})P(\theta_{\text{prop}})}{P(D \mid \theta_t)P(\theta_t)}$$

- Notice that the evidence constant $P(D)$ in the denominator cancels out!
- We only need to calculate the likelihoods and priors of the two states to determine the transition probability.

Proposal Distributions and Acceptance

How does the Metropolis-Hastings algorithm navigate the parameter space?

- **Proposal Distribution $q(\theta_{\text{prop}} \mid \theta_t)$:** A probability function used to suggest the next step. Typically a symmetric Gaussian centered on the current state:

  $$\theta_{\text{prop}} \sim \mathcal{N}(\theta_t, \sigma^2)$$

  Because it is symmetric, proposing θ_{prop} from θ_t has the same probability as proposing θ_t from θ_{prop}: $q(\theta_{\text{prop}} \mid \theta_t) = q(\theta_t \mid \theta_{\text{prop}})$.

- **Acceptance Probability (α):** The probability that we accept the proposed transition:

  $$\alpha = \min\left(1, \frac{P(D \mid \theta_{\text{prop}})P(\theta_{\text{prop}})}{P(D \mid \theta_t)P(\theta_t)}\right)$$

  - If the proposal increases the posterior, $\alpha = 1.0$ (we always accept).
  - If the proposal decreases the posterior, $\alpha < 1.0$ (we might accept).

Step-by-Step Metropolis-Hastings Algorithm

To run an MCMC chain:

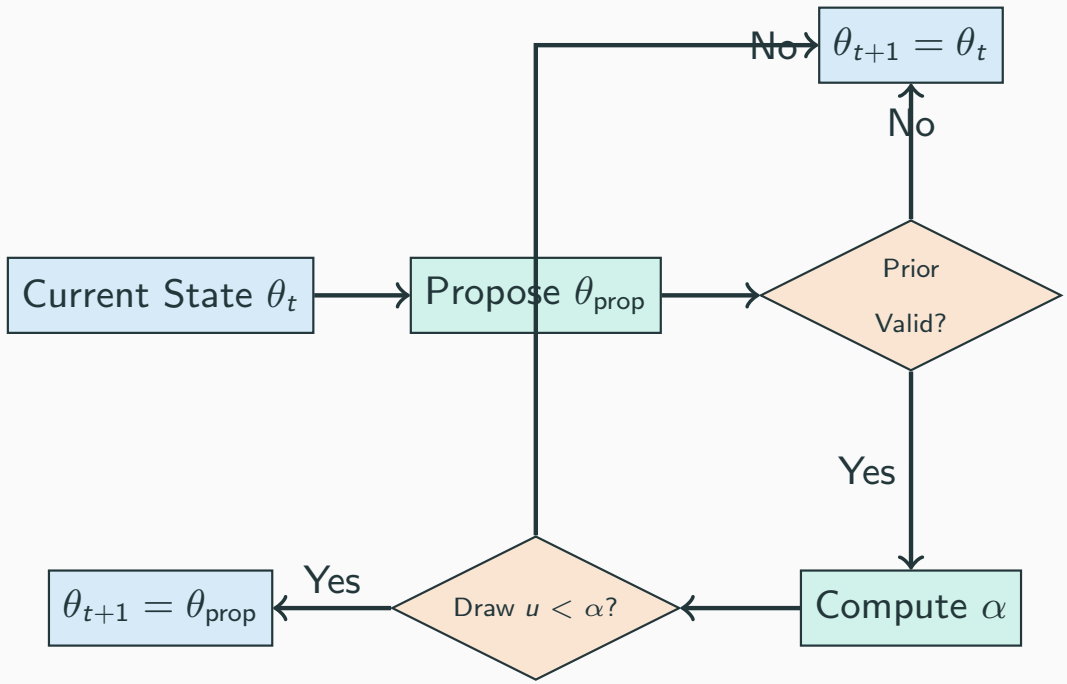
1. **Initialize:** Choose a starting parameter point θ_0 and a proposal step width σ.
2. **Loop:** For each step t from 0 to $M-1$:
   - Propose a candidate state: $\theta_{\text{prop}} \sim \mathcal{N}(\theta_t, \sigma^2)$.
   - Evaluate prior boundaries. If θ_{prop} is unphysical (outside priors), reject: set $\theta_{t+1} = \theta_t$ and skip to next step.
   - Calculate the acceptance probability:

$$\alpha = \min\left(1, \frac{\text{Likelihood}(\theta_{\text{prop}}) \cdot \text{Prior}(\theta_{\text{prop}})}{\text{Likelihood}(\theta_t) \cdot \text{Prior}(\theta_t)}\right)$$

   - Draw a random number $u \sim \mathcal{U}(0,1)$.
   - If $u < \alpha$: Accept proposal: set $\theta_{t+1} = \theta_{\text{prop}}$.
   - Else: Reject proposal: set $\theta_{t+1} = \theta_t$.

Metropolis-Hastings Decision Flowchart

Below is the step-by-step decision pathway executed at each iteration of the Metropolis-Hastings MCMC sampler:



Hand-Worked MCMC Step

Let's walk through a single MCMC parameter step:

- **Parameter:** Rate parameter θ.
- **Priors:** Flat uniform prior over $[0, 5]$.
- **Current State (t):** $\theta_t = 1.0$, with joint numerator score:

$$P(D \mid \theta_t)P(\theta_t) = 0.040$$

- **Step 1: Propose candidate θ_{prop}:** Draw a step from proposal distribution $\mathcal{N}(1.0, 0.05^2)$, yielding:

$$\theta_{\text{prop}} = 1.08$$

  Check prior: $1.08 \in [0, 5]$, so the proposal is valid.
- **Step 2: Calculate target score at proposal:**

$$P(D \mid \theta_{\text{prop}})P(\theta_{\text{prop}}) = 0.032$$

- **Step 3: Compute acceptance probability α:**

$$\alpha = \min\left(1, \frac{0.032}{0.040}\right) = \min(1, 0.80) = 0.80$$

- **Step 4: Draw random variable $u \sim \mathcal{U}(0, 1)$:**
  - If we draw $u = 0.54 \implies 0.54 < 0.80 \implies$ **Accept:** $\theta_{t+1} = 1.08$.
  - If we draw $u = 0.87 \implies 0.87 \geq 0.80 \implies$ **Reject:** $\theta_{t+1} = 1.00$.

MCMC Diagnostics: Burn-in and Autocorrelation

Before analyzing our MCMC samples, we must apply two preprocessing steps:

1. **Burn-In Phase:** The starting point θ_0 is often in a low-probability region. The early iterations show a systematic drift as the chain climbs toward the high-probability peak.
   - **Action:** Discard the first $10\% - 20\%$ of the chain (e.g., discard the first 1,000 steps of a 6,000-step chain). This ensures our analyzed samples only come from the converged stationary distribution.
2. **Autocorrelation (Thinning):** Because MCMC states depend on preceding steps, adjacent samples are correlated.
   - **Thinning:** We save only every k-th sample (e.g., keep every 5th or 10th sample) to reduce correlation and storage size, yielding independent draws from the posterior.

Trace Plots: Monitoring Convergence

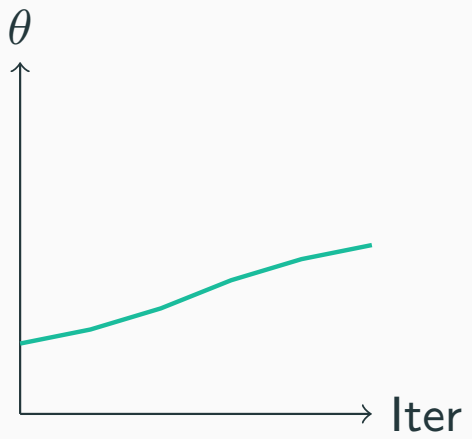
A **Trace Plot** displays the parameter value on the y-axis against the iteration step index on the x-axis. It is the primary diagnostic tool for monitoring chain convergence:

- **Caterpillar Look:** A healthy, converged MCMC chain looks like a tight, horizontal "fuzzy caterpillar." It oscillates rapidly around a stable mean value.
- **Poor Mixing (Drifting):** If the chain shows long-term trends or slow waves, it has poor mixing. This means the step size is too small or the sampler is stuck in a local region.
- **Tuning Proposal Width (σ):**
  - **Too Small:** Chain accepts almost every step but moves very slowly, failing to explore the parameter space.
  - **Too Large:** Chain proposes massive jumps into low-probability regions, resulting in almost all steps being rejected (chain gets stuck in horizontal lines).
  - **Optimal:** Aim for an acceptance rate of $\sim 20\% - 50\%$.

Trace Plot Configurations

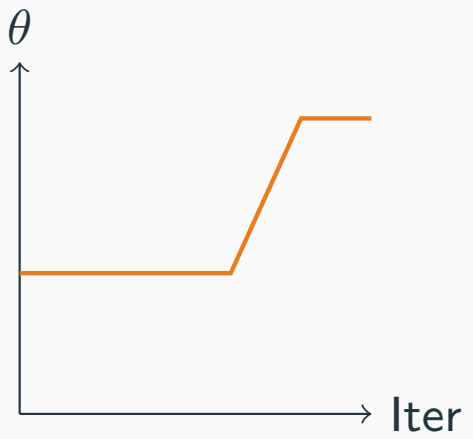
Below is a conceptual visualization of trace plots under different proposal widths:

Step Size Too Small



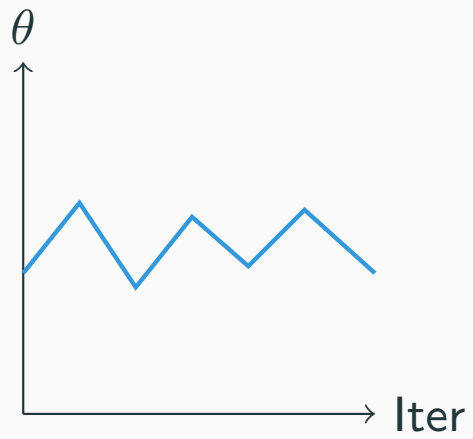
High acceptance, but slow drift. Fails to explore.

Step Size Too Large



High rejection rate. Long horizontal periods.

Optimal step size



Rapid oscillations around the mean. Healthy mixing.

Visualizing Posterior Marginals: Corner Plots

For multi-parameter models (like Type Ia coefficients A and B), we display MCMC results using a **Corner Plot**:

- **Diagonal Panels:** 1D histograms showing the marginal posterior distribution for each parameter. The peak represents the best fit, and the width represents the uncertainty.
- **Off-Diagonal Panels:** 2D contour maps or scatter plots showing the joint distribution of parameter pairs.
- **Mapping Degeneracies:** If two parameters are uncorrelated, the 2D contour is circular. If they are correlated, the contour is elongated (a degeneracy diagonal).
- The MCMC corner plot reveals how parameters trade off against each other, exposing degeneracies that point estimates (MLE) completely hide.

Part 3: Physics-Informed Machine Learning (PIML)

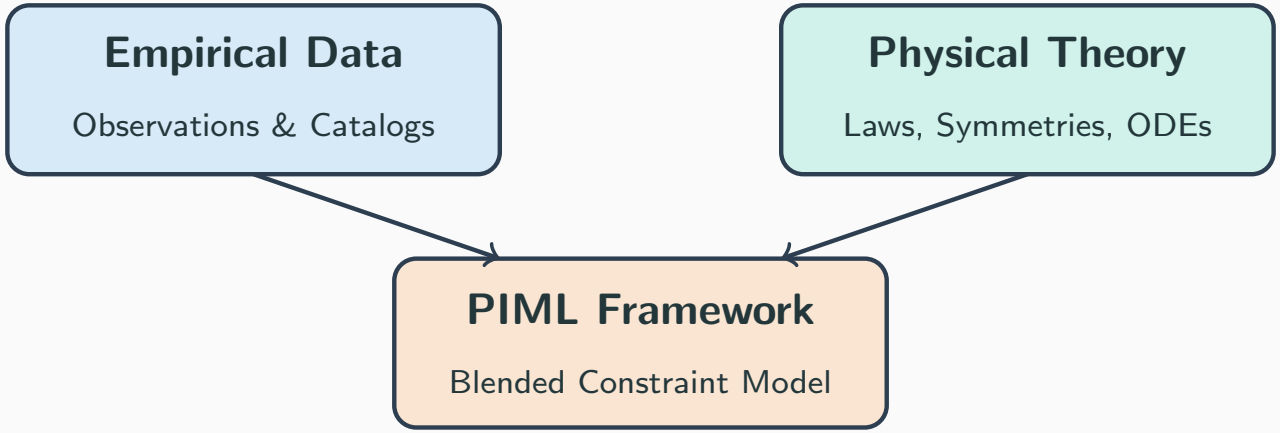
The Limits of Pure Machine Learning

Standard machine learning models (like multi-layer perceptrons, decision trees, or random forests) are purely data-driven. They excel at finding correlations, but fail in physical sciences due to key limitations:

1. **Physical Violations:** Data-driven models do not respect physical laws. A standard classifier might predict negative probabilities, violate conservation of energy, or output non-zero rates for mathematically impossible conditions.
2. **Out-of-Distribution Failure:** Deep neural networks are excellent interpolators but poor extrapolators. If tested outside the boundaries of the training set, their predictions decay rapidly into unphysical regimes.
3. **Massive Data Requirements:** Purely data-driven models require large training datasets to learn simple relationships (like $1/r^2$ gravity fields), which is problematic for rare astrophysical transients.
4. **Lack of Physical Interpretability:** Black-box models only yield relative feature sensitivities, not physical parameters, units, or rate coefficients.

What is Physics-Informed Machine Learning?

Physics-Informed Machine Learning (PIML) blends empirical observations (data) with prior scientific knowledge (theoretical models, symmetries, boundary conditions, or differential equations).



By constraining the search space of the model to only include physically valid functions, PIML:

- Restricts models to physically realistic boundaries.
- Reduces the volume of training data required by orders of magnitude.
- Generalizes and extrapolates reliably outside training regions.

Soft Constraints: Loss-Function Regularization

Consider a system governed by a differential equation:

$$f(\mathbf{x}, u(\mathbf{x})) = 0$$

where $u(\mathbf{x})$ is the physical quantity we wish to model, and $\mathbf{x}$ are inputs.

Composite Loss Function
To enforce this law, we define a composite loss:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \cdot \mathcal{L}_{\text{physics}}$$

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{i=1}^{N_d} \| u(\mathbf{x}_i) - u_{\text{observed},i} \|^2$$

$$\mathcal{L}_{\text{physics}} = \frac{1}{N_{\text{col}}} \sum_{j=1}^{N_{\text{col}}} \| f(\mathbf{x}_j, u(\mathbf{x}_j)) \|^2$$

The regularization coefficient $\lambda > 0$ balances data matching and physics compliance. This constraint is **soft** because the optimizer might accept small physical violations to better fit noisy training data.

Hard Constraints: Architectural Integration

Soft constraints can still violate physical boundaries if the loss optimization converges to a local minimum. To ensure physical laws are met exactly, we build them directly into the network architecture:

Examples of Architectural Integration

- **Non-Negativity Constraints:** If a physical quantity (like mass or rate) must be strictly non-negative, we pass the network's raw outputs through an exponential function:

$$u_{\text{physical}} = e^{u_{\text{raw}}} \quad \Longrightarrow \quad u_{\text{physical}} > 0 \text{ always}$$

- **Probability Conservation:** Enforces that predicted class assignments sum to 1.0 by applying a Softmax layer at the final output:

$$\sum_{c} P_c = 1.0 \text{ always}$$

- **Energy Conservation:** Hard-code an energy conservation block $E_{\text{kinetic}} + E_{\text{potential}} = E_{\text{total}}$ as an analytical layer.

Feature-Level Symmetries

We can also inject physical theory into machine learning models at the feature level:

Feature-Level (Input Integration): Transform input features to satisfy symmetries and physical invariances.

- **Example:** Enforce rotational invariance by converting Cartesian coordinates (x, y, z) to spherical coordinates (r, θ, ϕ) before training.

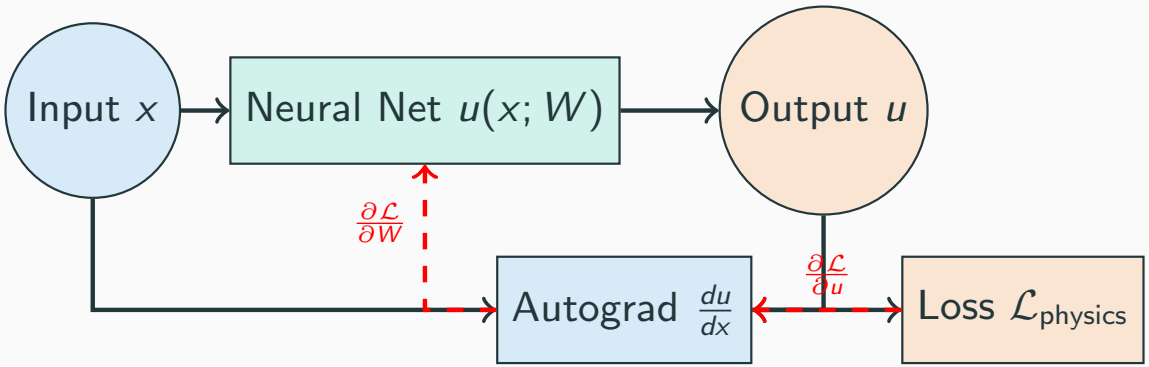
Automatic Differentiation (Autograd) in PIML

To evaluate differential equations (like $\frac{du}{dx}$) in a loss-level constraint, we must calculate the derivatives of our neural network's predictions with respect to its inputs.

- **Why not numerical differentiation?** Finite difference approximations ($\frac{u(x+\Delta x)-u(x)}{\Delta x}$) are computationally expensive (scale poorly with input dimension) and suffer from numerical truncation errors.
- **Why not analytical differentiation?** Calculating derivatives by hand for deep, non-linear networks is highly complex and impractical.
- **The PIML Solution: Autograd** Automatic differentiation tracks operations on a computational graph. It computes exact, analytical derivatives down to machine precision using the chain rule, scaling efficiently to high dimensions.

Autograd Computational Flow

Below is a schematic showing how automatic differentiation propagates gradients through a physical loss loop. Autograd calculates the exact physical derivative $\frac{du}{dx}$ and uses it to update the model weights W:



Part 4: Case Study: Supernova Rate Modeling (Task 13)

The Physics of Supernova Rates

In previous tasks, we compared host galaxy properties with background populations using statistical tests. Now, we define three competing models for supernova rates as functions of host galaxy properties:

1. **Null (Random) Model:** Supernovae are random events that do not depend on host properties:

$$R_{\text{Null}} = \text{const}$$

2. **Core-Collapse Supernova (CC-SN) Model:** Tracks instantaneous star formation as a power law of the Star Formation Rate (SFR):

$$R_{\text{CC}}(\text{SFR}) = \text{SFR}^{\beta}$$

where $\beta \approx 1.0$ if the rate is directly proportional to the active star formation.

3. **Type Ia Supernova Model:** The standard "A+B" model with delayed (stellar mass) and prompt (recent star formation) components:

$$R_{\text{Ia}}(M_\star, \text{SFR}) = A \cdot \left(\frac{M_\star}{10^{10} M_\odot} \right) + B \cdot \left(\frac{\text{SFR}}{M_\odot \ \text{yr}^{-1}} \right)$$

To simplify calculations and ensure numerical stability, we define:

$$R_{\text{Ia}}(M_\star, \text{SFR}) = A \cdot M_{\star,10} + B \cdot \text{SFR}$$

Formulating the Rate Likelihood

The probability of a specific galaxy i hosting a supernova out of a population of N galaxies is proportional to its relative rate:

$$P(\text{host}_i \mid \theta) = \frac{R_i(\theta)}{\sum_{j=1}^{N} R_j(\theta)}$$

For a sample of observed host galaxies, the joint likelihood is the product of their individual probabilities.

Negative Log-Likelihood (NLL) Formulation
To find the optimal parameters θ, we minimize the Negative Log-Likelihood ($\mathcal{L}$):

$$\begin{aligned}
\mathcal{L}(\theta) &= -\ln \prod_{i \in \text{observed}} P(\text{host}_i \mid \theta) \\
&= -\sum_{i \in \text{observed}} \ln \left(\frac{R_i(\theta)}{\sum_{j \in \text{background}} R_j(\theta)} \right) \\
&= -\sum_{i \in \text{observed}} \ln R_i(\theta) + N_{\text{observed}} \cdot \ln \left(\sum_{j \in \text{background}} R_j(\theta) \right)
\end{aligned}$$

Classical MLE point estimation

To fit our physical parameters (β for CC-SNe; A, B for Type Ia SNe) using point-estimate optimization:

- **Data Extraction:** Compile the observed host parameters ($M_\star$, SFR) and the background field galaxy parameters from the detected catalog.
- **L-BFGS-B Optimization:** Pass the custom NLL functions to a bounded minimizer (e.g., `scipy.optimize.minimize`).
- **Boundary Constraints:**
  - Enforce parameter bounds: $\beta \in [0.1, 3.0]$ to prevent physical collapse.
  - Enforce non-negativity: $A, B \in [0.0, 5.0]$ since supernova rates cannot be negative.
- **Output Estimates:** Recover point estimates, showing that CC-SNe trace SFR ($\beta \approx 1.0$), and Type Ia SNe depend on both mass and star formation ($A, B > 0$).

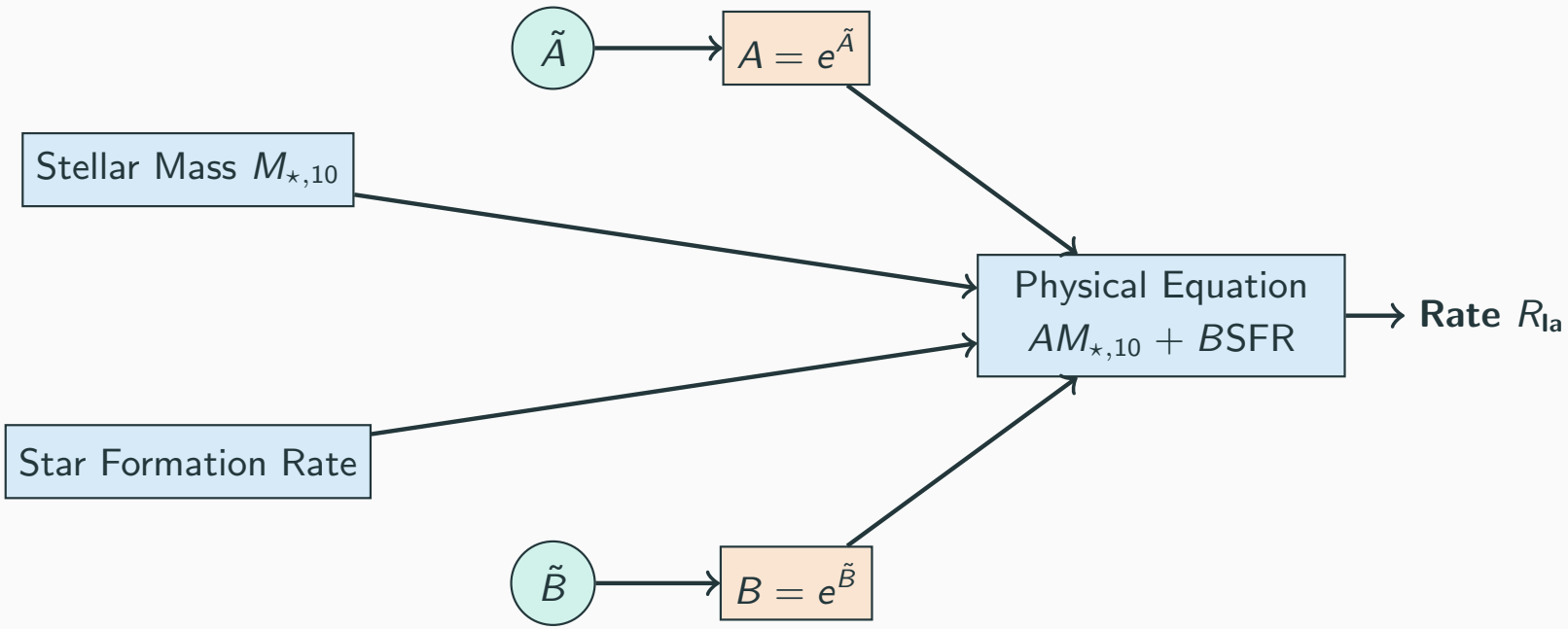
Constrained PINN Rate Model

Instead of classical numerical solvers, we can recover parameters using a PyTorch neural network that incorporates our physical rate equations:

- **Constraining Parameters:** We define parameters (β, A, B) as neural network weights. To enforce strict positive boundaries $(A, B, \beta > 0)$ without coordinate clipping, we parameterize their raw inputs and apply an exponential layer:
$$\beta = e^{\tilde{\beta}}, \quad A = e^{\tilde{A}}, \quad B = e^{\tilde{B}}$$
- **Forward Propagation:** The network takes the host properties as inputs and computes the rates directly using the physical equations:
$$R_{\mathrm{Ia}} = A \cdot M_{\star,10} + B \cdot \mathrm{SFR}$$
- **Optimization Loss:** The model minimizes the joint Negative Log-Likelihood loss using the Adam optimizer. Gradients flow from the loss directly back to the physical weights $(\tilde{\beta}, \tilde{A}, \tilde{B})$ via automatic differentiation.

PINN Rate Model Architecture

Below is the network architecture for our Task 13 PINN. Unbounded parameters $(\tilde{A}, \tilde{B})$ are transformed via exponential layers to satisfy the physical positivity bounds of the rate equation:



Model Validation & Goodness-of-Fit

How do we prove that our physical models are superior to the null model?

- **Synthetic Catalog Generation:** Using the fitted MLE parameters, we calculate rate weights $w_i = R_i / \sum R_j$ for every galaxy in the detected mock universe catalog. We sample 5,000 galaxies with replacement using these weights.
- **Goodness-of-Fit Testing:** We run 2-sample Kolmogorov-Smirnov (K-S) tests on the physical distributions of the simulated catalogs against the observed hosts:
  - **Null Model Comparison:** Compares observed hosts against randomly sampled non-host galaxies. Yields p-values $\ll 10^{-5}$, rejecting the null hypothesis.
  - **Physics Model Comparison:** Compares observed hosts against physics-weighted mock galaxies. Yields p-values > 0.05 (e.g., $p \approx 0.50$), confirming we cannot distinguish our model's predictions from reality.

Part 5: Case Study: Convolved DTD Solvers (Task 15)

The Physics of Type Ia Supernovae

Type Ia supernovae originate from binary systems where a carbon-oxygen white dwarf reaches the Chandrasekhar mass limit or undergoes a double white dwarf merger.

- **Delay Time (τ):** The time elapsed between the formation of the star system (starburst) and the subsequent supernova explosion.
- **Delay Time Distribution ($\Psi(\tau)$):** The distribution of delay times for a single-burst stellar population.
- **Progenitor Power Law:** Theoretical white-dwarf merger models suggest that the DTD follows a power law:

$$\Psi(\tau) \propto \tau^{-\gamma}$$

where $\gamma \approx 1.0 - 1.2$ for delay times $\tau \geq t_{\text{min}} \approx 100$ Myr.

Convolutions with Star Formation History (SFH)

A real galaxy is not a single stellar population. It forms stars continuously over cosmic time.

Convolved Rate Model
The supernova rate in a galaxy at age t is the convolution of its historical Star Formation History (SFR(t)) with the DTD (Ψ):

$$R_{\mathrm{Ia}}(t) = \int_0^t \mathrm{SFR}(t-\tau)\Psi(\tau)d\tau = \int_0^t \mathrm{SFR}(t-\tau)\tau^{-\gamma}d\tau$$

Analytical Star Formation History Model
We approximate the SFH using a standard delayed-exponential model:

$$\mathrm{SFR}(t \mid \tau_{\mathrm{SF}}) = \mathrm{SFR}_0 \cdot \frac{t}{\tau_{\mathrm{SF}}} e^{-t/\tau_{\mathrm{SF}}}$$

where τ_{SF} is the characteristic star formation timescale.

Discretizing the Convolution Integral

To evaluate convolved rates numerically, we map each galaxy to a cosmic time grid $t_k = k \cdot \Delta t$ from $t = 0$ to the galaxy's age $T \approx 13.7$ Gyr:

Discretized Rate Equation

$$R_{\mathrm{Ia}}(T \mid \tau_{\mathrm{SF}}, \gamma) = \sum_{\tau_k \geq t_{\mathrm{min}}}^{T} \mathrm{SFR}(T - \tau_k \mid \tau_{\mathrm{SF}}) \cdot \tau_k^{-\gamma} \cdot \Delta t$$

- We define the delay time grid step $\Delta t = 0.1$ Gyr.
- $\mathrm{SFR}(T - \tau_k \mid \tau_{\mathrm{SF}})$ represents the star formation rate of the galaxy at the lookback time τ_k.
- $\tau_k^{-\gamma}$ represents the fraction of stars born at lookback time τ_k that explode today.
- The total rate is the sum over all lookback time steps.

Reconstructing γ via PyTorch Optimization

We recover the power-law slope γ from a sample of observed hosts by optimizing a PyTorch convolved rate solver:

- **Parameterization:** Define γ as an exponential parameter to enforce physical boundaries: $\gamma = e^{\tilde{\gamma}}$.
- **PyTorch Solver:** Evaluates the vectorized numerical convolution across the training galaxies:
  1. Computes the SFR grid matrix for all galaxies based on their age and $\tau_{\rm SF}$.
  2. Computes the DTD vector $\boldsymbol{\psi}(\gamma) = \tau^{-\gamma}$.
  3. Performs matrix multiplication to evaluate rates: $\mathbf{r} = \mathbf{S}\boldsymbol{\psi} \cdot \Delta t$.
- **Loss Function Optimization:** Minimizes the Negative Log-Likelihood:

$$\mathcal{L} = -\sum_{i \in \text{hosts}} \ln R_i + N_{\text{hosts}} \ln \left(\sum_{j \in \text{all}} R_j \right)$$

  using the Adam optimizer. The solver successfully recovers the true power-law slope $\gamma \approx 1.15$.

Below is the computational pipeline of our convolved solver. Gradients flow from the loss function through the numerical convolution block back to the physical parameter γ:

Diagnostics and DTD Validation Results

Once optimization completes, we evaluate the DTD reconstruction:

- **Parameter Convergence:** The optimizer successfully updates γ from its initialization toward the true physical power-law slope of $\gamma = 1.15$.
- **DTD Visual Check:** Plotting the true DTD ($\tau^{-1.15}$) against the recovered DTD ($\tau^{-\gamma_{\text{recovered}}}$) shows excellent visual agreement across delay times $0.1 \leq \tau \leq 3.0$ Gyr.
- **Loss Convergence Curve:** The loss history displays a smooth decay curve, confirming stable optimization gradients throughout training epochs.

PIML Summary & Checklist

When designing Physics-Informed Machine Learning models, follow these guidelines:

PIML Design Guidelines

1. **Identify Symmetries and Slices:** Incorporate rotational, translational, or scale invariances into feature engineering before model fitting.
2. **Enforce Bounds Architecturally:** If parameters must be positive, use exponential layers (e^{θ}) to enforce this constraint exactly.
3. **Balance Composite Losses:** If using soft constraints, monitor the loss terms $\mathcal{L}_{\text{data}}$ and $\mathcal{L}_{\text{physics}}$ separately to ensure one does not dominate the other.
4. **Validate Physically:** Check model predictions using independent physical criteria (e.g., K-S tests or energy conservation conservation checks).